Name: Priyanshi Singh
Roll No: 2401115
T22
Subject: ASD&D

EXPERIMENT 1
Version Control Operations Using Git

Aim

To learn and perform version control operations using Git by creating a local repository, tracking file changes, managing commits, creating branches, merging branches, and viewing project history.

Theory

Git is a distributed version control system developed by Linus Torvalds in 2005. It helps developers track changes in source code, collaborate with team members, maintain different versions of a project, and recover previous versions when needed.
Version control records every change made to files in a repository. Git stores these changes as snapshots called **commits**. Developers can work on separate **branches**, merge their work into the main project, and synchronize repositories using remote services like GitHub.

Advantages of Git

- Tracks change efficiently.
- Supports multiple developers working simultaneously.
- Allows rollback to previous versions.
- Maintains complete project history.
- Facilitates collaboration through branching and merging.
- Works offline because it is a distributed version control system.

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Software: Git
- Terminal: Command Prompt, PowerShell, Git Bash, or Terminal
- Optional: GitHub account

Commands Used

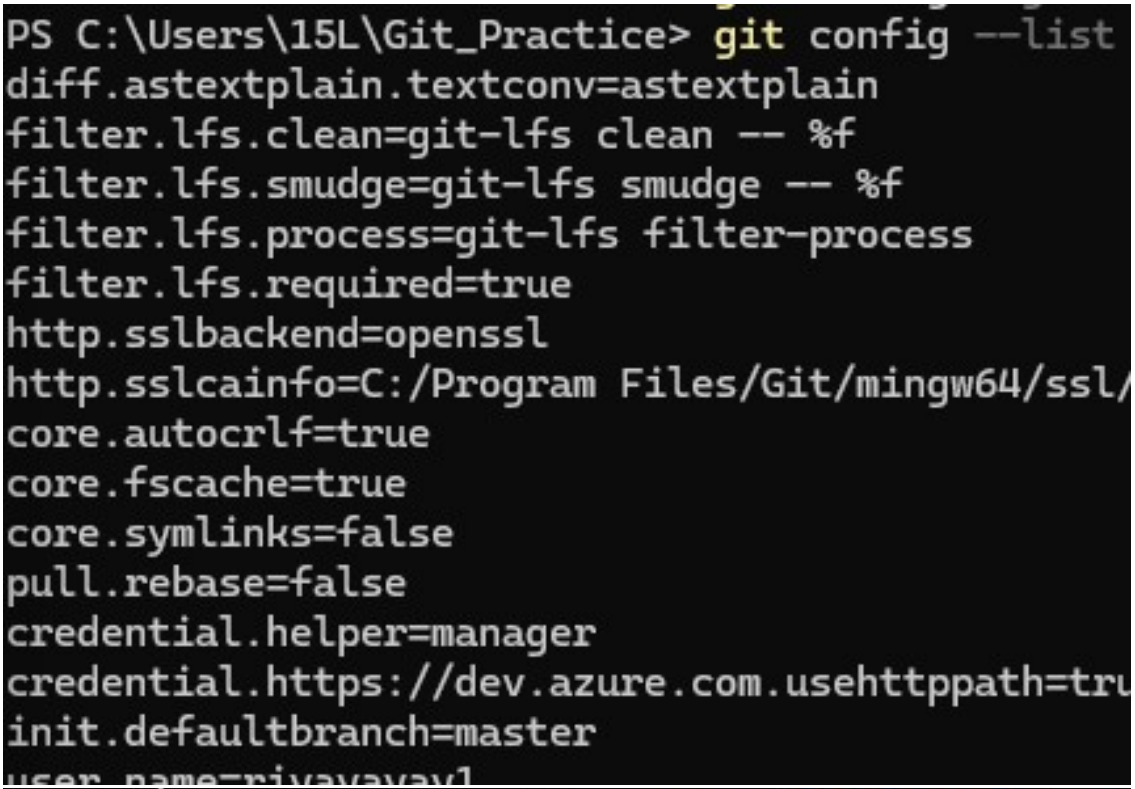
Create a table like this:

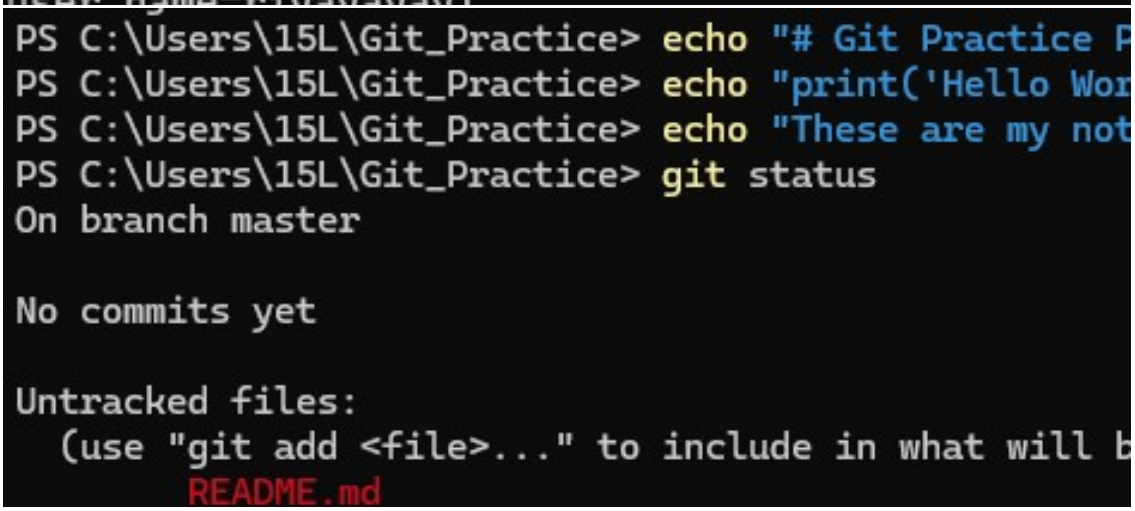
Name: Priyanshi Singh
Roll No: 2401115
T22
Subject: ASD&D

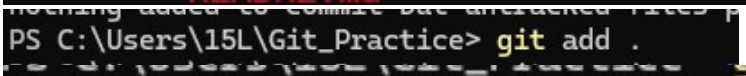
Sr. No.	Command	Purpose
1	git --version	Displays installed Git version.
2	git config --global user.name	Sets Git username.
3	git config --global user.email	Sets Git email address.
4	git config --list	Displays Git configuration.
5	git init	Initializes a new Git repository.
6	git status	Shows repository status.
7	git add .	Stages all files.
8	git commit -m "message"	Saves staged changes.
9	git log	Displays commit history.
10	git log --oneline	Displays compact commit history.
11	git diff	Shows differences between file versions.
12	git branch	Lists or creates branches.
13	git switch feature	Switches to another branch.
14	git merge feature	Merges a branch into the current branch.
15	git mv	Renames a file.
16	git rm	Deletes a tracked file.
17	git ls-files	Lists tracked files.
18	git reflog	Shows reference log.
19	git show	Displays details of a commit.
20	git remote -v	Lists remote repositories.
21	git restore <file>	Discards local changes made to a file and restores it to the last committed version.
22	Git stash	Temporarily saves uncommitted changes without creating a commit.
23	Git stash pop	Restores the most recently stashed changes and removes them from the stash list.
24	Git branch -d	Deletes a branch that has already been merged.
25	git clone <repourl>	Creates copy

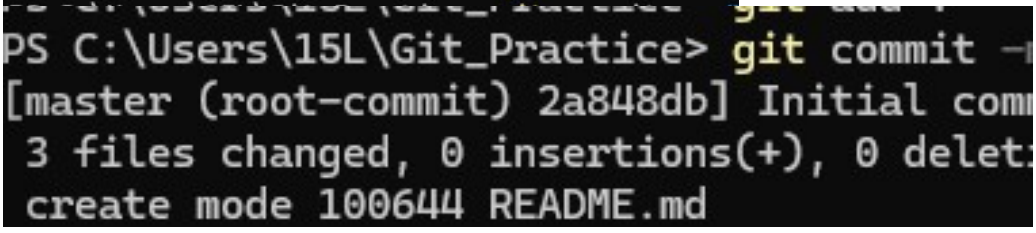
Name: Priyanshi Singh
Roll No: 2401115
T22
Subject: ASD&D

Output









Name: Priyanshi Singh
Roll No: 2401115
T22
Subject: ASD&D

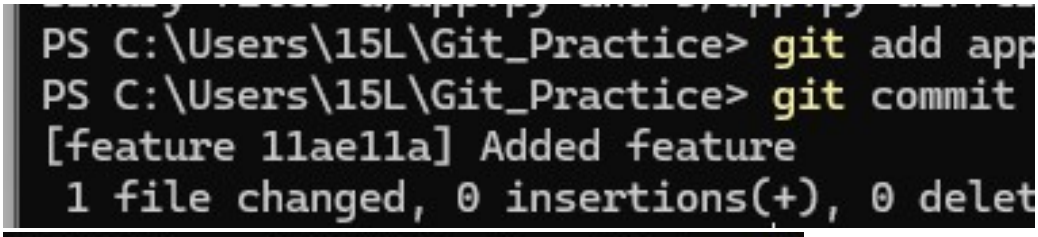
```
PS C:\Users\15L\Git_Practice> git log
commit 2a848db60b7621a1d4d97191d873394631c3ca
Author: riyayayay1 <you@example.com>
Date:   Wed Jul 15 13:35:41 2026 +0530
PS C:\Users\15L\Git_Practice> gi
```

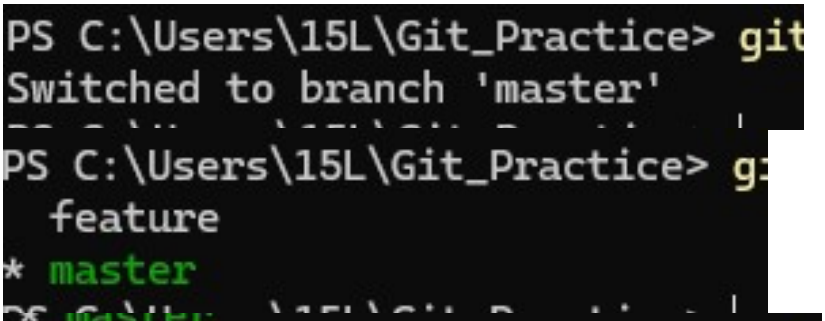
```
PS C:\Users\15L\Git_Practice> echo "Git is easy t
PS C:\Users\15L\Git_Practice> git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be
  (use "git restore <file>..." to discard changes
no changes added to commit (use "git add" and/or "
PS C:\Users\15L\Git_Practice> git diff
diff --git a/notes.txt b/notes.txt
index fa39f4b..267b8ac 100644
PS C:\Users\15L\Git_Practice> git add notes.txt
PS C:\Users\15L\Git_Practice> git commit -m "Upda
[master 7167db6] Updated notes
 1 file changed, 0 insertions(+), 0 deletions(-)
PS C:\Users\15L\Git_Practice>
```

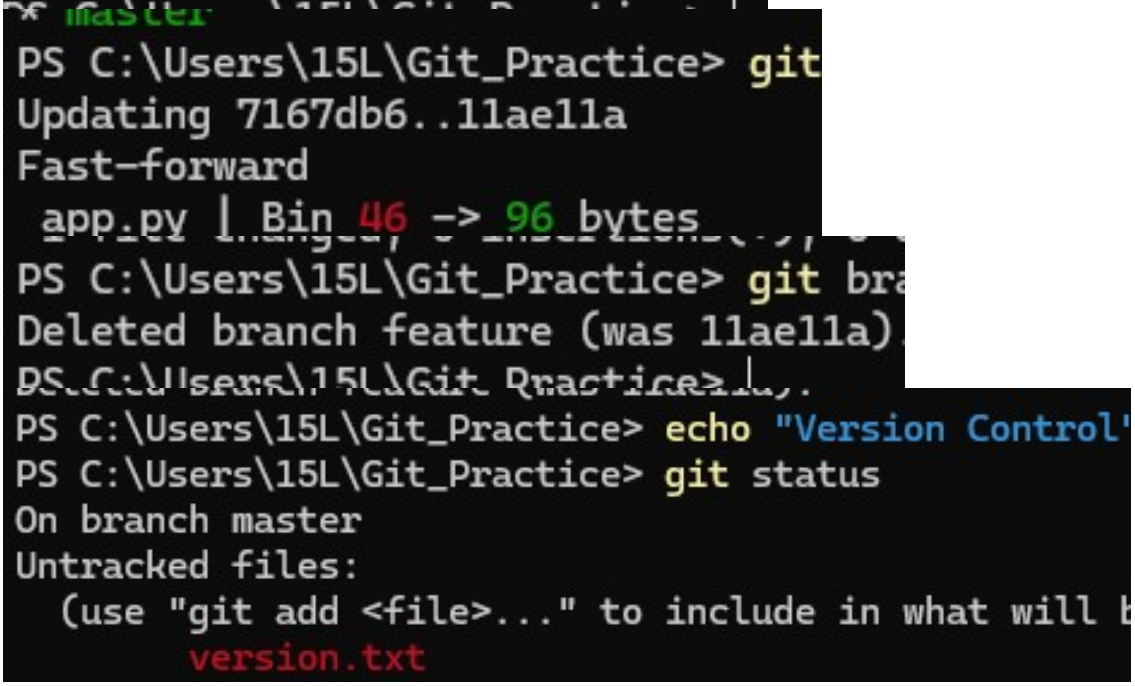
```
PS C:\Users\15L\Git_Practice> git
PS C:\Users\15L\Git_Practice> git
* master
PS C:\Users\15L\Git_Practice> git
Switched to branch 'feature'
```

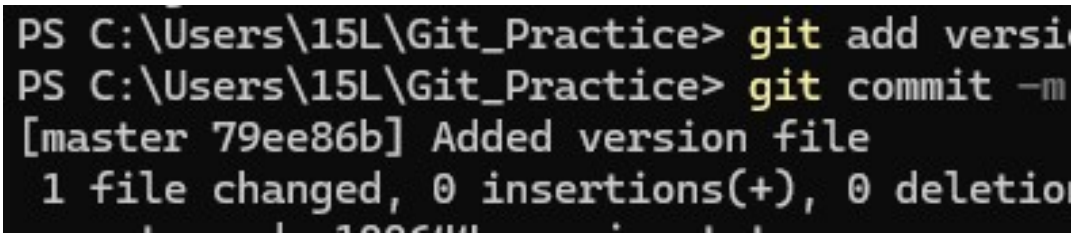
```
PS C:\Users\15L\Git_Practice> echo "print('Feature
PS C:\Users\15L\Git_Practice> git diff
diff --git a/app.py b/app.py
index b978e17..a5d54d8 100644
Binary files a/app.py and b/app.py differ
```

Name: Priyanshi Singh
Roll No: 2401115
T22
Subject: ASD&D









Name: Priyanshi Singh
Roll No: 2401115
T22
Subject: ASD&D

```
PS C:\Users\15L\Git_Practice> git mv version.t
PS C:\Users\15L\Git_Practice> git commit -m "R
[master de458de] Renamed version file
```

```
 rename version.txt => git_version.txt (100%)
PS C:\Users\15L\Git_Practice> git rm git_versio
rm 'git_version.txt'
PS C:\Users\15L\Git_Practice> git commit -m "De
[master bd39d68] Deleted version file
 1 file changed, 0 insertions(+), 0 deletions(-
 delete mode 100644 git_versio
PS C:\Users\15L\Git_Practice>
README.md
app.py
```

```
PS C:\Users\15L\Git_Practice> git reflog
bd39d68 (HEAD -> master) HEAD@{0}: commit: D
de458de HEAD@{1}: commit: Renamed version fi
79ee86b HEAD@{2}: commit: Added version file
11ae11a HEAD@{3}: merge feature: Fast-forwar
7167db6 HEAD@{4}: checkout: moving from feat
11ae11a HEAD@{5}: commit: Added feature
```

Name: Priyanshi Singh
Roll No: 2401115
T22
Subject: ASD&D

```
PS C:\Users\15L\Git_Practice> git log
* bd39d68 (HEAD -> master) Deleted ver
* de458de Renamed version file
* 79ee86b Added version file
* 11ae11a Added feature
* 7167db6 Updated notes
* 2a848d0 Initial commit
PS C:\Users\15L\Git_Practice> git show
commit bd39d689b03cefec3e5568986c46be15c894
Author: riyayayay1 <you@example.com>
Date:   Wed Jul 15 13:44:08 2026 +0530

    Deleted version file

diff --git a/git_version.txt b/git_version.
```

```
PS C:\Users\15L\Git_Practice> g
```

```
PS C:\Users\15L\Git_Practice> # Save uncommitted
PS C:\Users\15L\Git_Practice> git stash
No local changes to save
PS C:\Users\15L\Git_Practice>
PS C:\Users\15L\Git_Practice> # Restore stashed
PS C:\Users\15L\Git_Practice> git stash pop
No stash entries found
```

Result

Successfully performed version control operations using Git, including repository creation, file tracking, commits, branching, merging, and history management.

Conclusion

Through this practice , I learned how to use Git as a version control system to manage project files efficiently. I learned to initialize a Git repository, configure Git, track file

changes, stage and commit updates, view commit history, create and switch branches, merge branches, rename and delete tracked files, and restore previous versions when needed. I also understood how Git maintains project history and supports organized development, making it easier to collaborate and manage software projects.